

Penyelesaian Tugas 8: REST API dengan Django Ninja

Dokumen ini berisi langkah-langkah dan kode lengkap untuk menyelesaikan Tugas 8 berdasarkan materi "REST API dengan Django Ninja".

Langkah 1: Memperbarui Library di requirements.txt

Tambahkan library django-ninja ke dalam file requirements.txt proyek Anda agar dapat digunakan untuk membangun REST API.

File: requirements.txt

```
django
psycopg2-binary
pillow
django-silk
django-ninja
```

Catatan: Setelah mengubah requirements.txt, pastikan Anda menginstal ulang dependencies (misalnya dengan menjalankan `docker-compose down` dan `docker-compose up --build -d` jika menggunakan Docker, atau `pip install -r requirements.txt` jika menggunakan virtual environment lokal).

Langkah 2: Membuat Endpoint API (GET, POST, PUT, DELETE)

Buat sebuah file baru di dalam direktori aplikasi inti (misalnya `core/apiv1.py`). Di dalam file ini, kita akan membuat *instance* dari `NinjaAPI` dan mendefinisikan beberapa endpoint untuk Kalkulator, pengelolaan User, serta fitur Registrasi dengan skema validasi.

File: core/apiv1.py

```
from ninja import NinjaAPI, Schema
from django.contrib.auth.models import User
from pydantic import validator
import re
```

```
# Inisialisasi Django Ninja API
```

```
apiv1 = NinjaAPI()
```

```
# -----
```

```
# 1. Endpoint Basic GET & POST
```

```
# -----
```

```
@apiv1.get('hello/')
```

```

def helloApi(request):
    return "Menyala abangkuh ..."

@apiv1.post('hello/')
def helloPost(request):
    if 'nama' in request.POST:
        return f"Selamat menikmati ya {request.POST['nama']}"
    return "Selamat tinggal dan pergi lagi"

```

```

# -----
# 2. Endpoint Kalkulator (GET & POST)
# -----

```

```

@apiv1.get('calc/{nil1}/{opr}/{nil2}')
def calculator(request, nil1: int, opr: str, nil2: int):
    hasil = nil1 + nil2
    if opr == '-':
        hasil = nil1 - nil2
    elif opr == 'x':
        hasil = nil1 * nil2
    return {'nilai1': nil1, 'nilai2': nil2, 'operator': opr, 'hasil': hasil}

```

Skema untuk Validasi POST Kalkulator

```

class Kalkulator(Schema):
    nil1: int
    nil2: int
    opr: str
    hasil: int = 0

    def calcHasil(self):
        hasil = self.nil1 + self.nil2
        if self.opr == '-':
            hasil = self.nil1 - self.nil2
        elif self.opr == 'x':
            hasil = self.nil1 * self.nil2
        return hasil

```

```

@apiv1.post('calc')
def postCalc(request, skim: Kalkulator):
    skim.hasil = skim.calcHasil()
    return skim

```

```

# -----
# 3. Endpoint Mock PUT & DELETE Users

```

```
# -----
@apiv1.put('users/{id}')
def userupdate(request, id: int):
    # Mengambil nilai 'nama' dari body request jika ada
    nama_baru = request.POST.get('nama', 'User Baru')
    return f"User dengan id {id} Nama aslinya adalah Herdiono kemudian diganti menjadi {nama_baru}"
```

```
@apiv1.delete('users/{id}')
def userDelete(request, id: int):
    return f"Hapus user dengan id: {id}"
```

```
# -----
# 4. Skema Validasi dan Endpoint Registrasi User
# -----
```

```
# Skema Input
```

```
class Register(Schema):
```

```
    username: str
    password: str
    email: str
    first_name: str
    last_name: str
```

```
@validator("username")
```

```
def validate_username(cls, value):
```

```
    if len(value) < 5:
        raise ValueError("Username harus lebih dari 4 karakter")
    return value
```

```
@validator('password')
```

```
def validate_password(cls, value):
```

```
    if len(value) < 8:
        raise ValueError("Password harus lebih dari 8 karakter")
    pattern = r'^(?=.*[A-Za-z])(?=.*\d).+$'
    if not re.match(pattern, value):
        raise ValueError("Password harus mengandung huruf dan angka")
    return value
```

```
# Skema Output (Response)
```

```
class UserOut(Schema):
```

```
    id: int
    username: str
    first_name: str
```

```
last_name: str
email: str
```

```
@apiv1.post('register/', response=UserOut)
```

```
def register(request, data: Register):
```

```
    """
```

Endpoint untuk registrasi pengguna dengan validasi inputan:

- username: minimal terdiri dari 5 karakter

- password: minimal terdiri dari 8 karakter dan harus mengandung huruf dan angka

```
    """
```

```
    newUser = User.objects.create_user(
```

```
        username=data.username,
```

```
        password=data.password,
```

```
        email=data.email,
```

```
        first_name=data.first_name,
```

```
        last_name=data.last_name
```

```
    )
```

```
    return newUser
```

Langkah 3: Mendaftarkan API ke URL Proyek (Routing)

Selanjutnya, hubungkan apiv1 yang baru dibuat dengan konfigurasi URL utama aplikasi Django agar endpoint tersebut dapat diakses.

File: simplelms/urls.py

```
from django.contrib import admin
```

```
from django.urls import path
```

```
from core.apiv1 import apiv1 # Import instance apiv1 dari aplikasi core
```

```
urlpatterns = [
```

```
    path('admin/', admin.site.urls),
```

```
    # ... url path lainnya ...
```

```
    # Daftarkan base URL untuk Django Ninja API
```

```
    path('api/v1/', apiv1.urls),
```

```
]
```

Langkah 4: Menjalankan dan Menguji API

1. Jalankan server pengembangan Django:
python manage.py runserver 0.0.0.0:8009

(Atau jalankan container docker Anda jika memakai Docker).

2. Buka browser dan arahkan ke: <http://localhost:8009/api/v1/docs>.
Halaman antarmuka interaktif (Swagger UI) akan terbuka dan menampilkan seluruh endpoint yang telah Anda buat (GET /hello, GET /calc, POST /register, dll). Anda dapat langsung menguji setiap endpoint dari halaman tersebut.